ELI — Code Mode Execution Layer

ELI v2.2.3

August 2026

Contents

Blueprint — Code-Mode Execution Layer for ELI (gap analysis)

Status: draft. Date: 2026-06-11. REVISED after auditing the codebase: an earlier version of this doc proposed building machinery ELI already has. This version is a GAP ANALYSIS — what exists, and the small genuine delta — not a greenfield design. Original ELI work throughout; the only outside idea is the general principle of deterministic gating, which ELI already implements its own way.

BUILT (2026-06-11)

Core implemented, gated, and tested (17 tests, `tests/test_code_mode.py`): - **Restricted exec (the one new piece):** `eli/coding/restricted_exec.py` — Full-Control-gated (`is_full_control()`, the GUI toggle), AST-whitelist (10 escape classes rejected: import/ eval/open/getattr/dunder-attr/__import__/def/lambda/with/exec), restricted namespace. - **Facade (sugar):** `eli/tools/api.py` — `api.call("ACTION", **args)` + helpers, proxying to `execute()`; `api.actions()` for discovery. - **Runner:** `eli/coding/code_mode.py` — generate→gate+validate+run→retry loop (injectable `generate`, testable without a model). - **Lane redirect:** `CODE_SOLVE` with `args.code_mode=True` runs in-process against `eli.api` (fail-closed: refuses unless Full Control is ON). - **#4 retrieval:** confirmed ABSENT (no top-K capability retriever); v1 passes a bounded `api.actions()` list — a semantic retriever is a future refinement, not required.

Remaining (deferred, behaviour-policy): the router auto-selecting code-mode for multi-step requests under Full Control — needs live validation, so it's flag-reachable for now.

---

0. Correction up front

"Code-mode" (the model writes a small program against a curated API instead of emitting MCP-style tool-calls) is the token-frugal, composable, offline-friendly direction the field is moving to — and the right fit for ELI's 8 GB / small-context reality. **But ELI already has ~85% of the substrate.** This doc exists to name the ~15% that's actually new, so we build only that.

1. What ELI ALREADY HAS (audited, with files)

Capability the blueprint needs	Already in ELI	File
Code-agent loop: plan → implement → verify → run → retry	**`CodeAgent.solve()`** — "orchestrator that composes ELI's coding capabilities"	`eli/coding/agent.py`
Planner / implementer separation	`plan_task()`, `implement()`	`eli/coding/planner.py`
Candidate verification + **auto-synthesised tests**	`verify_candidate()`, `synthesize_tests()`	`eli/coding/verification.py`
Tree search over candidates	`tree_search()`	`eli/coding/search.py`
Learn from failed attempts	`BugMemory`	`eli/coding/bug_memory.py`
DAG decomposition of a task	`solve_dag()`	`eli/coding/plan_graph.py`
Bounded sandbox execution (CPU limit, scrubbed env, kill-switch)	`run_code()`	`eli/coding/sandbox.py`
Deterministic risk gate (action classes → approved/blocked/pending-confirm)	`approval_engine.evaluate_record()`	`eli/runtime/approval_engine.py`
Fail-closed command allowlist + destructive-pattern block (`rm -rf /`, `mkfs`, `dd`...)	`security` + `_BLOCKED_PATTERNS`	`eli/runtime/security.py`, `executor_enhanced.py`
Authority / autonomy posture	`authority_gate`, `operator_policy` (proposal_only/supervised/...)	`eli/runtime/authority_gate.py`, `eli/execution/operator_policy.py`
AST validate-and-retry of generated code	`code_examiner` (used live 2026-06-11)	`eli/runtime/code_examiner.py`
Capability **registry** (register/get/list)	`capability_registry`	`eli/tools/registry/capability_registry.py`
Capability **doc/digest generator** from the manifest	`generate_capabilities_doc()`	`eli/tools/registry/capabilities_doc.py`
Offline enforcement / crisis guard	`netguard`, `crisis_guard`	`eli/core/`
Reasoning-native planning (model thinks, then emits)	working post-`e7ab0b9`	`eli/cognition/gguf_inference.py`

Conclusion: ELI is not missing "a code agent," "a sandbox," or "a safety gate." It has all three, plus verification, search, bug-memory, a risk engine, a command allowlist, and a capability registry. The CodeAgent already does CodeAct-style plan→code→verify→sandbox→retry.

2. The genuine DELTA (the only new work)

The existing `CodeAgent` solves **coding tasks** — produce a function/script as a *deliverable*. Code-mode-as-assistant-execution needs it pointed at a different target: the model writes code that **calls ELI's own capabilities** to fulfil a *conversational request*, and the reply is synthesised from the execution result. Each candidate delta was audited IN FULL (2026-06-11); statuses below are grounded with file:line evidence.

1. **`eli.api` facade — MOSTLY EXISTS (only typed sugar is new).** The callable dispatch already exists: `execute(action, args)` at `executor_enhanced.py:10653` runs any of the 216 actions by name.

Generated code can already call `execute("SUMMARIZE_FILE", {...})`. The *only* delta is a typed, discoverable wrapper (`eli.api.files.summarize(path)`) generated from the registry — convenience + discoverability over an existing surface, not new execution machinery.

2. **In-process AST-whitelist + restricted exec — GENUINELY NEW (the one real piece).** `sandbox.run_code()` (`coding/sandbox.py:106`) is **process-isolated** (subprocess, scrubbed env, CPU limit) and runs *standalone* code — it deliberately cannot reach ELI's live state (the loaded model, memory stores). Code-mode must run **in-process** to call live `eli.api/execute()`, so it **cannot reuse that sandbox**. So an in-process restricted-exec is required: an AST pass allowing only `eli.api.*/execute(...)` + safe builtins, run in a restricted-globals namespace. It **composes** existing parts — the `code_examiner` AST walker + `approval_engine` action-classes for the GREEN/AMBER/RED decision — but the in-process restricted-exec itself is new. This is THE work.
3. **Routing lane to the code agent — ALREADY EXISTS as `CODE_SOLVE`.** Router lane (`router_enhanced.py:2254`, `_CODE_SOLVE_RE`) + executor handler (`executor_enhanced.py:8702`) → `eli.coding.solve` (plan→search→verify→repa quick/ thorough, DAG, background). Reuse it. The only variant is targeting `eli.api` for assistant-actions rather than producing a standalone script, + grounded synthesis of the result (reuses the `540f514` no-confabulation work). Do NOT build a new lane.
4. **Per-query capability retrieval — UNCONFIRMED; parts exist, no retriever found.** There is a manifest (`capability_inventory.generated.json`), a digest generator (`capabilities_doc.generate_capabilities_` `capability_sync`, and a FAISS store — but a top-K *capability retriever* was NOT located in the audit. Verify before building; if absent, it's a small assembly from the manifest + existing vector store, not new infra.

Net after full audit: #3 exists, #1 is sugar over an existing dispatch, #4 is unconfirmed (parts present), and only **#2 (the in-process restricted-exec) is genuinely new** — and it composes existing AST + gate machinery. The real build is: one in-process AST-whitelist + the typed facade, then point the existing `CODE_SOLVE` path at `eli.api`.

3. Phased plan (composing what exists)

- **Phase 0 — the one new piece: in-process restricted-exec (~3–5 days).** AST whitelist (reuse `code_examiner`'s walker) allowing only `eli.api.*/execute(...)` + safe builtins; run in a restricted-globals namespace IN-PROCESS (not `sandbox.run_code()`, which is process-isolated and can't see live state); classify each call via `approval_engine` action classes for the GREEN/AMBER/RED decision. This is the genuinely new safety surface.
- **Phase 1 — facade + point CODE_SOLVE at it (~2–3 days).** Generate the typed `eli.api` wrapper from the registry over `execute()`. Add a CODE_SOLVE *variant* (or arg) that targets `eli.api` for assistant-actions and feeds the result into grounded synthesis — reusing the existing lane, agent, quick/thorough modes, and background handling. No new routing lane, no new agent.
- **Phase 2 — retrieval (if needed) + harden (~few days).** Add top-K capability retrieval ONLY if the audit confirms it's absent; add code-mode cases to the eval harness; add the live integration test the suite is missing.

Total honest estimate: **~1–1.5 weeks**, and the bulk of it is the single new piece (the in-process restricted-exec). The agent loop, sandbox, risk gate, verification, bug-memory, registry, AND the `CODE_SOLVE` routing lane are already yours.

4. Verified in full (2026-06-11)

Audited against the project directory, not assumed: - **Callable dispatch surface?** YES — `execute(action, args)` at `executor_enhanced.py:10653`. Generated code can already run any of the 216 actions by name. - **Routing lane to the code agent?** YES — `CODE_SOLVE` (`router_enhanced.py:2254` + `executor_enhanced.py:8702` → `eli.coding.solve`). Already plan→search→verify→repair, quick/thorough, DAG, background. - **Sandbox: namespace-restricted or process-only?** PROCESS-only (`coding/sandbox.py:106`: subprocess + scrubbed env + CPU limit), for *standalone* code. **Cannot host live-state code-mode** → the in-process AST-whitelist (§2.2) is genuinely required. - **Risk gate to compose with?** YES — `approval_engine.evaluate_record()` (action classes → approved/blocked/pending-confirm) + the command allowlist. Map `eli.api` calls onto it. - **AST walker to reuse?** YES — `code_examiner` (used live this session). - **Existing `eli.api`-style typed facade under `eli/tools/`/`eli/plugins/`?** NOT found — the registry stores metadata, not a callable typed surface. This (small) piece is new. - **Top-K capability retriever?** NOT found — manifest + digest generator + FAISS exist, but no retriever. Verify once more at build time; if truly absent, assemble from existing parts.

5. What stays uniquely ELI

Voice, vision, gaze, hard-offline, emergent persona, and the entire coding-agent + gate stack you already built — untouched. This is wiring ELI's own parts into a new lane, not importing anyone's design.